Scrapebot: Project Overview

Background, goals, technical stack and how the system works

Date	12 October 2026
Describes	The code on branch **main** after pull request #9 (commit ec72fb9). Parts that are not built yet are marked **Planned**.
Figures from	The full run of 9 October 2026 (813 links, 808 stores, 82 minutes, LLM on) and the roadmap in the repository. Each figure carries its date.
Source of truth	The Markdown documents in **docs/**. Where this PDF and they disagree, the Markdown wins.

1. Summary

Scrapebot is an internal tool for the Wooden Ships wholesale team. It finds knitwear stores, visits each store's public website, and collects its products, prices, brands, wholesale pages and contacts exactly as the store wrote them. Every run ends with a **final list**: the stores that sell several brands and sell knitwear, with their knitwear products. A separate step then sorts the stores into possible partners, competitors and existing customers.

- **Why it exists:** store research was done by hand, one store at a time, and version 1 of the bot read only Shopify stores well.
- **How it reads a store:** through seven layers, from the cheapest and most exact (the platform's own product feed) to the most expensive (an LLM). It stops looking for products at the first layer that finds any.
- **What it costs:** reading stores is free. Only store discovery and the LLM cost money, and each has a hard cap. The LLM cost US$0.13 for the 808-store run of 9 October.
- **What it never does:** get past bot protection, log in, ignore `robots.txt`, or store HTML.
- **Where it stands:** discovery, bulk input, the seven layers, eight output formats, the final list, the web interface and a first wholesale analysis are built. The main open problem is reach: on 9 October 287 of 808 stores were read (section 8).

2. Background

2.1 The problem

Wooden Ships sells knitwear to stores. To grow that business the team has to know which stores could stock the brand and which ones compete with it. Before this project, four things stood in the way:

Problem	Effect
Store research was manual, one store at a time	Slow, and the results were not uniform from one store to the next
Version 1 read only Shopify stores well	Stores on other platforms often came back with zero products
Input and output were tied to CSV with fixed columns	A list in another format had to be reshaped by hand first
Version 1 was designed for one region, the United States	No choice of country or language

2.2 What the first survey showed

On 12 August 2026 version 1 ran over 67 store domains in Naples, Florida. The result set the direction for version 2:

Finding	Count
Shopify stores with an open `/products.json` feed	26
Products read from those feeds, with prices	6,167
Non-Shopify sites (custom, Wix, WooCommerce, BigCommerce, Squarespace)	41
Homepages that answered HTTP 403 (H&M, Macy's, Four Seasons: national chains, not prospects)	3
Homepages with a broken TLS certificate	2

No real prospect was lost to bot detection. The stores could be reached; they were hard to *read*, because most non-Shopify boutiques publish no machine-readable product data and version 1 understood one format only. Version 2 closes that gap in steps: the cheapest and most exact method first, a browser only when a page needs one, and an LLM as the last resort.

3. Goals and Boundaries

3.1 End goal

Decide which stores are competitors and which are worth approaching as wholesale partners. Scrapebot supplies the data and a first sorting for that decision. Contacting the stores is outside the tool.

Part	Status
Store discovery	Built, from the command line and the web interface
Bulk input in any format	Built
Raw data acquisition (seven layers)	Built
Export to many formats	Built for 8 formats. Google Sheets and PostgreSQL are planned
Final list: multi-brand stores that sell knitwear	Built
Wholesale analysis: partner, competitor or existing customer	First version built (`scrapebot analyze`). Three of its four input files are still missing (section 8)
Change detection between runs	Planned (milestone M5)
Reachout (email, CRM)	Out of scope

3.2 Business outcomes

Outcome	How it is measured
Less research time	Time from "link list ready" to "data ready for analysis". The target is set once today's manual time has been measured
Wider coverage	Share of non-Shopify stores that yield product data
Faster decisions	The data already holds the signals that separate competitors from partners
Used by the team	Runs per month by the team, not only by the bot's author

3.3 Release targets

Metric	Target	Type
Links in = processed + skipped, in the run report	100% of runs	Hard
Stores with status `ok` and zero products	0	Hard
Products with a source and an evidence URL	100%	Hard
API keys that reach logs or output files	0	Hard
Requests that break `robots.txt` or get past a challenge	0	Hard
Non-Shopify stores that yield at least 1 product	60% or more	Proposed
Accuracy of LLM-read products on a hand-checked set of 20 sites	90% or more	Proposed
Duration for 300 domains without the browser and the LLM	60 minutes or less	Proposed
Runs per month by the team	2 or more	Proposed

Hard targets must hold for the release. Proposed targets are confirmed after the survey milestone (M1).

3.4 Out of scope

- Getting past bot protection: no CAPTCHA solving, no proxy rotation, and `robots.txt` is respected.
- Logging in to a site. Public pages only.
- Storing or exporting HTML. Page text and structured data are stored instead.
- Opening Instagram or Facebook profiles. They are read only from search results.
- Sending email or feeding a CRM.
- Accounts and permissions. The interface is an internal tool that runs locally.

4. How It Works

4.1 Two entry points, one engine

Entry point	Command	For whom
Web interface	`make serve`, then http://127.0.0.1:8765	Operators: find or paste links, preview, scrape, download. Pages: Scrape, Overview, Deteksi (who blocks the bot), Traffic toko, Riwayat run
Command line	`uv run scrapebot run <input>`	Developers and automation (n8n, cron)

Both build the same run configuration and call the same pipeline, so a run gives the same data either way. The pipeline never imports interface code. The interface text is in Indonesian.

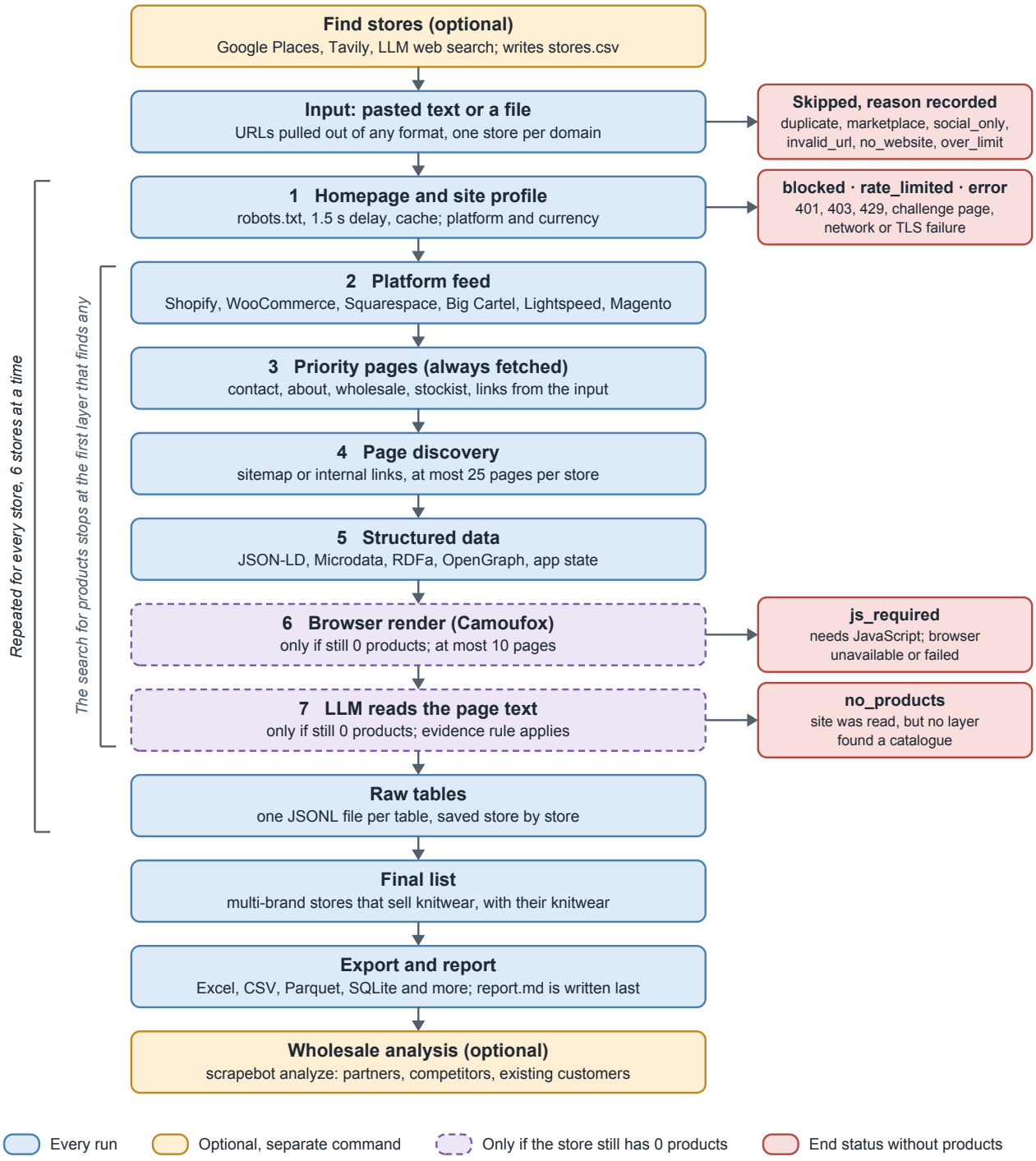


Figure 1. From a list of links to the files the team downloads. Layers 1 to 7 are the core of the system.

4.2 The steps

Step	What happens
Find stores (optional)	`scrapebot discover`, or the tab *Cari toko otomatis*, searches Google Places, the web through Tavily (including Instagram and Facebook profiles that appear in search results) and an LLM with web search. Hits for the same store merge into one row. Every source has a request cap or a cost cap, and answers are cached. It never opens a store's website. The result is `stores.csv`.
Input	Links are pasted as free text or uploaded as txt, csv, tsv, xlsx, json, jsonl or parquet. URLs are pulled out of any text and grouped by domain. Other columns travel along untouched. A skipped link keeps its reason, so links in always equals processed plus skipped. At most 1,000 links per run by default.
1. Homepage and site profile	One fetch of the front page: `robots.txt` is checked, each domain waits 1.5 seconds between requests, and successful answers are cached for 7 days. The platform and the currency are read from the page. A 401, 403, 429 or a challenge page ends the store as blocked.
2. Platform feed	Shopify, WooCommerce, Squarespace, Big Cartel, Lightspeed and Magento 2 hand out their product list as JSON. This is free and exact. On 9 October, 96% of all products came from a feed.
3. Priority pages	Contact, about, wholesale and stockist pages are always fetched, feed or no feed, so contacts are not missed.
4. Page discovery	For a store without a feed: product and collection pages are found through the sitemap or by following internal links. At most 25 pages per store.
5. Structured data	Products are read from the data a page embeds for search engines and for its own scripts: JSON-LD, Microdata, RDFa, OpenGraph and app state.
6. Browser render	Only for a store that still has no products. Camoufox, a Firefox build, renders pages that need JavaScript and captures the JSON they load. Same rules as plain HTTP. It is not used to get past challenges. At most 10 pages per store.
7. LLM	Only for a store that still has no products. The model receives the text of up to 6 pages. A product is kept only if its page was one of those sent, its title is in that page's text and its price is a number on that page. Every row is flagged `needs_review`. On by default with `openai/gpt-4o-mini` and a US$1 budget per run; without a key the run goes on and the report says so.
Raw tables	Rows are appended store by store, so an interrupted run loses nothing and can be resumed. Values stay as found: a price written as "€ 63,95" is stored as "€ 63,95".
Final list	A store is on it when it was read, sells three or more outside brands (none above 90% of its products) and has at least one knitted garment or accessory. The LLM judges a knitwear store whose brands leave the type unclear. Nothing is removed from the raw tables.
Export and report	Every chosen format is written, then `report.md`: the reconciliation line, statuses, products by source, the final list, LLM tokens and cost. The report is written last, so its presence marks a finished run.
Wholesale analysis (optional)	`scrapebot analyze <run>` joins a finished run with the team's files (stockists, Salesforce accounts, brands, price points). Each store gets a segment and a 0 to 100 score with its reasons. It never fetches a store.

4.3 How a store ends

Status	Meaning
`ok`	At least one product was found
`no_products`	The site was read, but no layer found a catalogue
`js_required`	The site needs JavaScript and the browser was unavailable or failed
`blocked`	401, 403, 429 or a challenge page from the store. Not bypassed
`rate_limited`	The server network limited our address. That network is paused and the store is tried once more at the end of the run
`error`	Network failure, another HTTP error, or disallowed by `robots.txt`
Skipped at input	`duplicate`, `marketplace`, `social_only`, `invalid_url`, `no_website`, `over_limit`

5. Technical Stack

What the code uses today, taken from the project's dependency files.

Part	Tools	Used for
Language and packaging	Python 3.11 to 3.13, uv, hatchling	The bot, its command line and the local API
Models and configuration	pydantic, pyyaml	Run configuration, table rows, YAML config files
HTTP	requests, urllib.robotparser, own file cache	The one fetcher all network access goes through; no `Accept-Language` header, so prices stay in the store's base currency
Input	urlextract, tldextract, openpyxl, pyarrow	Finding URLs in any text, grouping by domain, reading Excel and Parquet
HTML and structured data	beautifulsoup4, lxml, extruct, chompjs	Links and page text; JSON-LD, Microdata, RDFa, OpenGraph; JavaScript objects
Browser	Camoufox (Firefox driven through Playwright)	Rendering pages that need JavaScript
LLM	litellm, instructor, trafilatura, python-dotenv	One interface for every provider, validated structured answers, main page text, keys from .env
Store discovery	Google Places Text Search, Tavily, LLM web search via litellm	Finding stores before a run
Output	json, csv, sqlite3 (standard library), openpyxl, pyarrow, duckdb	JSON, JSONL, CSV, TSV, Excel, Parquet, SQLite, DuckDB
Local API	FastAPI, uvicorn, python-multipart	Runs in the background, live progress, downloads, on 127.0.0.1 only
Web interface	React 19, TypeScript, Vite, lucide-react, openapi-typescript	The operator's interface; its types are generated from the API
Own-store traffic	Shopify Admin GraphQL API (ShopifyQL)	The Traffic toko page: live sessions on our own store
Analysis	OpenStreetMap Nominatim	Turning a postal code into a point, once, cached
Tests and quality	pytest, ruff, pyright, pre-commit; Vitest, Testing Library, Playwright	All tests run offline on recorded responses; `make check` must pass before a commit

Chosen but not in the code yet: httpx with hishel, tenacity, aiolimiter and protego (to replace requests), pycountry, babel, phonenumbers and lingua-language-detector (region settings), gspread and SQLAlchemy (Google Sheets and PostgreSQL).

6. What a Run Produces

Each run has its own folder, and that folder is the source of truth:

```
data/runs/<run_id>/
├── config.json      run settings, never API keys
├── tables/*.jsonl   raw tables
├── export/          the chosen formats; final/ and knit/ inside
├── analysis/        written by scrapebot analyze
├── summary.csv      one row per link
├── manifest.json    package versions, duration, row counts
└── report.md        written last
```

Table	One row per	Holds
`runs`	run	Configuration without keys, version, times
`inputs`	input link	The original link, its status or skip reason, the whole input row
`stores`	store	Status, platform, currency, layers tried, product and knitwear counts, store type and where it came from, brands, the identity the store declares
`products`	product	Title, raw price, currency, vendor, product type, tags, URL, source, evidence URL, review flag, knitwear flags, the full source object
`pages`	fetched page	Page kind, HTTP status, HTTP or browser, the page text
`contacts`	contact found	Email, phone, Instagram, Facebook, TikTok, LinkedIn or Pinterest, with the page it came from
`llm_calls`	LLM call	Model, prompt version, tokens, cost
`changes`	change between runs	Empty until change detection is built
`final_stores`, `final_products`	store or product on the final list	Derived: brands, knitwear counts, contacts, wholesale pages; knitwear only
`knit_products`	knitwear product of any store read	Derived: with the store's name, type and whether it is on the final list

Every table joins on `run_id` and `domain`.

7. Rules the Bot Keeps

- **No access control is bypassed.** `robots.txt` is respected over HTTP and in the browser. A store that refuses is recorded as blocked.
- **Public pages only.** No login.
- **Polite pace.** One request at a time per server, 1.5 seconds apart; a network that limits us is paused, not pushed.
- **Raw means raw.** Values are stored as found, with their source and an evidence URL. Prices are not converted.
- **HTML is never stored or exported.** Page text is.
- **Every link is accounted for.** Links in = processed + skipped.
- **Keys stay out of the data.** API keys live in the session or in .env, never in logs, output files, reports or the cache.
- **Discovery stays on search results.** It does not open store websites or social profiles.
- **Tests never touch the network.**

8. Where It Stands

8.1 The full run of 9 October 2026

Measure	Result
Links in	813 = 808 stores visited + 5 skipped (4 marketplace, 1 social only)
Duration	82 minutes
Stores read (ok)	287
Stores not read	375 blocked, 116 no products, 26 error, 4 need JavaScript
Products	195,901, of which 2,794 are flagged for review
Products by source	Shopify feed 159,583 (81%); Lightspeed feed 24,231; other feeds 3,713; structured data 7,347; LLM 812; browser JSON 215
Contacts	4,256 (1,111 phones, 1,073 Instagram, 992 emails)
Final list	190 stores with 22,582 knitwear products
Not on the final list	521 not read; 58 read but without knitwear; 39 sell their own label
LLM	201 calls, US$0.13. It found products for 25 of 145 stores sent, and judged the type of 56 stores
Wholesale analysis	217 retail partners, 2 B2B partners, 28 competitors, 8 existing customers

Reach is the main open problem. 375 of 808 stores ended as blocked. On the run of the day before, 306 of 376 blocked stores were not refusing the bot: Shopify's shared network was limiting our address after many requests. The bot has since learned to tell the two apart (status `rate_limited`, a pause, one retry). That was checked on 6 stores, not yet on a full list.

8.2 Built and planned

Built	Planned
Store discovery; bulk input; the seven layers; 8 output formats; run report with reconciliation; stop and resume; final list and knitwear-only download; web interface with the Deteksi and Traffic toko pages; wholesale analysis; input templates and backups	Change detection between runs; region settings for any country, phone parsing per region, page language detection; Google Sheets input and output; PostgreSQL output; the survey and benchmark on a hand-checked set of 20 sites (M1); the five handover documents and version 2.0.0 (M6)

8.3 Waiting on

- **Analysis inputs:** the Salesforce account export, the list of peer and competitor brands, and Wooden Ships' price points. Without them existing accounts are recognised only through the stockist list, and the brand and price points are left out of the score.
- **B2B discovery:** today's lists hold almost no hotel, resort, club or museum shops, because discovery searched for knitwear stores. They need their own search queries.
- **Google Places:** there is no API key yet, so this source has never run against the real service. Tavily and the LLM agent have: the discovery of 9 October found 62 stores, 60 of them with a website.
- **A full run with the rate-limit handling**, to see how many of the blocked stores become readable.